



中国石油大学(北京)

计算机科学与技术 专业

课程设计报告

课程设计名称：编译原理课程设计

学 院 人工智能学院

专 业 计算机

班 级 23-3 23-1

姓 名 郭宇杰, 张珊

指导教师 朱雪峰

目录

学生实践类课程学习日志.....	4
1 编译原理课程设计报告：实践 1.....	6
1.1 Flex 词法分析.....	6
1.1.1 目的和任务.....	6
1.1.2 项目结构和文件功能.....	6
1.1.3 实现过程.....	7
1.1.4 构建和运行.....	9
1.1.5 结果与结论分析.....	9
1.2 Bison 语法分析.....	10
1.2.1 目的和任务.....	10
1.2.2 项目结构和文件功能.....	10
1.2.3 在实践 1.1 基础上的修改和新增内容.....	11
1.2.4 主要函数和模块说明.....	12
1.2.5 语法规则和错误恢复.....	13
1.2.6 构建和运行.....	13
1.2.7 结果与结论分析.....	14
2 编译原理课程设计报告：实践 2 语义分析.....	15
2.1 目的和任务.....	15
2.2 项目结构和文件功能.....	15
2.3 在实践 1 基础上的修改和新增内容.....	17
2.4 主要数据结构和函数功能.....	17
2.4.1 类型系统.....	17
2.4.2 字段、符号和作用域.....	18
2.4.3 AST 辅助函数.....	18
2.4.4 语义检查主流程函数.....	18
2.4.5 错误输出函数.....	19
2.5 语义检查内容.....	19
2.6 构建和运行.....	20
2.7 结果与结论分析.....	20
3 编译原理课程设计报告：实践 3 中间代码生成.....	22
3.1 目的和任务.....	22
3.2 项目结构和文件功能.....	22
3.3 在前两次实践基础上的修改和新增内容.....	24
3.4 主要数据结构和函数功能.....	24
3.4.1 IR 类型系统.....	24
3.4.2 字段、符号、作用域和函数签名.....	24
3.4.3 输出和临时对象生成.....	25
3.4.4 声明、类型和函数签名收集.....	25
3.4.5 语句和表达式翻译.....	26

3.4.6 IR 生成总入口.....	26
3.5 IR 指令格式.....	26
3.6 构建和运行.....	27
3.7 结果与结论分析.....	27
4 编译原理课程设计报告：实践 4 目标代码生成.....	29
4.1 目的和任务.....	29
4.2 项目结构和文件功能.....	29
4.3 在前三次实践基础上的修改和新增内容.....	30
4.4 主要数据结构和函数功能.....	31
4.4.1 MIPS 后端数据结构.....	31
4.4.2 IR 解析和变量收集函数.....	31
4.4.3 操作数装载和存储函数.....	31
4.4.4 指令翻译函数.....	32
4.4.5 MIPS 生成总入口.....	32
4.5 目标代码生成策略.....	32
4.6 构建和运行.....	33
4.7 结果与结论分析.....	33
5 编译原理课程设计报告：实践 5 中间代码优化.....	34
5.1 目的和任务.....	34
5.2 目录结构.....	34
5.3 构建与测试方法.....	35
5.4 源码文件和函数功能.....	35
5.4.1 `src/iropt_main.c`.....	35
5.4.2 `src/iropt.h`.....	36
5.4.3 `src/iropt.c`.....	36
5.5 修改和添加内容.....	36
5.6 优化方法.....	37
5.6.1 常量传播和复制传播.....	37
5.6.2 常量折叠.....	37
5.6.3 局部公共子表达式消除.....	37
5.6.4 无用赋值删除.....	37
5.7 结果与结论.....	37
6 结果与结论分析.....	39

中国石油大学（北京）人工智能学院

学生实践类课程学习日志

学生姓名：郭宇杰 学号：2023011636 班级：计算机 23-3 班

学生姓名：张玥 学号：2022011528 班级：计算机 23-1 班

课程名称：编译原理课程设计

日期	主要问题和解决方案	指导教师
6.28	主要问题：实验环境依赖 Flex、Bison、GCC、SPIM、Docker/IRSim 等工具，且 C-- 文法资料较分散。解决方案：先整理课程资料和目录结构，编写环境检查脚本，明确各阶段输入输出和测试入口，为后续实践统一基础。	朱雪峰
6.29	主要问题：词法规则中整数、浮点数、非法标识符和注释规则容易相互冲突，块注释还需要处理嵌套和未闭合情况。解决方案：按最长匹配和规则优先级调整 Flex 正则，使用 COMMENT 状态处理块注释，并统一输出 A 类词法错误。	朱雪峰
6.30	主要问题：Flex 与 Bison 衔接时 token 属性、语法树节点和错误恢复需要保持一致，默认语法错误容易产生连锁报错。解决方案：增加 PARSER_BUILD 条件编译、设计 AST 节点结构，设置表达式优先级，并在局部产生式中加入错误恢复规则。	朱雪峰
7.1	主要问题：语义分析需要同时处理作用域、类型等价、函数声明定义关系和结构体/数组访问，错误表达式还可能导致重复报错。解决方案：建立类型系统和符号表，按 AST 分层检查语义规则，使用错误类型抑制级联报错，并在分析结束后统一检查未定义函数。	朱雪峰
7.2	主要问题：中间代码生成中左值地址和右值求值容易混淆，数组、结构体、函数调用和控制流翻译也需要统一到线性 IR。解决方案：为表达式分别实现求值和取地址逻辑，用临时变量与标签生成三地址代码，并借助 IRSim 对生成结果进	朱雪峰

	行运行验证。	
7.3	主要问题：MIPS 后端需要正确实现函数栈帧、参数传递、返回地址保存、递归调用和间接访存，参数顺序处理不当会导致运行结果错误。解决方案：基于 IR 统一收集变量和临时变量，采用栈帧存储策略，按 ARG/PARAM 规则处理调用，并用 SPIM 验证公开样例和手写样例。	朱雪峰
7.4	主要问题：IR 优化既要减少冗余代码，又不能跨控制流边界或间接访存错误传播，公共子表达式和无用赋值删除存在语义风险。解决方案：以基本块为单位进行常量传播、复制传播、常量折叠和公共子表达式消除，并结合活跃变量分析保守删除无副作用赋值，用 IRTsim 比较优化前后输出。	朱雪峰
7.5	主要问题：最终材料需要同时体现实现过程、测试结果和运行截图，多个分报告合并时还要保证分页、标题层级、图片和 PDF 格式正确。解决方案：补充各实践的构建、批测和代表性样例截图，合并 1.1 到 5 的报告并统一标题样式，最后转换为 PDF 进行页数和格式检查。	朱雪峰

1 编译原理课程设计报告：实践 1

课程设计名称：编译原理课程设计

实践内容：实践 1.1 学习 Flex, 用 Flex 实现词法分析

实践 1.2 学习 Bison, 用 Bison 实现语法分析

1.1 Flex 词法分析

1.1.1 目的和任务

本次实践在 `submission/proj1` 目录中独立完成，目标是根据 C++ 语言词法规则，使用 GNU Flex 实现词法分析程序。程序读取 `.cmm` 源文件后，将字符流识别为词法单元；遇到不符合词法规则的输入时，输出 A 类词法错误、错误行号和说明文字。

本阶段主要任务如下：

1. 阅读 `course/Appendix.pdf` 和 `course/Project\_1.pdf`，整理 C++ 语言需要识别的 token。
2. 使用 Flex 编写 `src/lexer.l`，识别关键字、标识符、整数、浮点数、运算符和分隔符。
3. 过滤空白字符、`//` 单行注释和 `/\* ... \*/` 块注释。
4. 对未定义字符、非法整数、非法浮点数、非法标识符和非法注释输出 A 类词法错误。
5. 编写 `Makefile` 和 `tools/run\_lexer\_tests.sh`，使实践 1.1 可以在 `proj1` 目录中单独构建、单独演示。

1.1.2 项目结构和文件功能

实践 1.1 相关文件均位于 `submission/proj1` 下，目录结构如下：

```
submission/proj1/
├─ Makefile
├─ course/
│   └─ Appendix.pdf
│   └─ Project_1.pdf
├─ docs/
│   └─ practice1.1-lexer.md
│   └─ practice1.1-lexer-report.md
├─ src/
│   └─ lexer.l
│   └─ lexer_main.c
│   └─ token.h
├─ test/
│   └─ proj1-2-3-6-exp1-1.cmm
│   └─ proj1-2-3-7-exp1-0.cmm
│   └─ proj1-2-3-7-exp5-0.cmm
│   └─ proj1-test1.cmm
└─ tools/
    └─ check_env.sh
    └─ run_lexer_tests.sh
```

各文件功能如下：

- `Makefile`：实践一独立构建入口。`make lexer` 生成词法分析器，`make lexer-demo` 运行词法公开样例，`make demo` 同时运行词法和语法演示。
- `src/lexer.l`：Flex 词法规则文件，是实践 1.1 的核心实现。它定义 token 正则表达式、注释处理状态、错误处理动作，并在独立 lexer 模式下返回 `token.h` 中的 token 枚举。
- `src/lexer\_main.c`：独立词法分析程序入口。它负责打开输入文件、调用 `yylex()` 循环扫描、根据 `CMM\_LEX\_DUMP` 决定是否打印 token 流，并在存在词法错误时返回非零状态。

- ``src/token.h``: 独立 lexer 模式使用的 token 枚举和 ``lexical_error_count`` 声明, 保证 ``lexer.l`` 和 ``lexer_main.c`` 使用同一套 token 类型。
- ``tools/check_env.sh``: 检查本实践需要的 ``flex``、``bison``、``gcc`` 等工具是否存在。
- ``tools/run_lexer_tests.sh``: 运行 ``Project_1.pdf`` 中与词法相关的公开样例, 并直接打印源文件路径和程序输出, 便于人工对照验收。
- ``test/proj1-2-3-6-exp1-1.cmm``: ``Project_1.pdf`` 2.3.6 必做样例 1, 检查未定义字符的 A 类错误。
- ``test/proj1-2-3-7-exp1-0.cmm``: ``Project_1.pdf`` 2.3.7 选做样例 1, 检查八进制和十六进制整数 token。
- ``test/proj1-2-3-7-exp5-0.cmm``: ``Project_1.pdf`` 2.3.7 选做样例 5, 检查单行注释和块注释过滤。
- ``test/proj1-test1.cmm``: 手写综合样例, 可用于现场展示 token 流。

1.1.3 实现过程

1.1.3.1 Token 和正则表达式设计

``src/lexer.l`` 中先定义常用正则片段:

```
DIGIT      [0-9]
NZDIGIT    [1-9]
LETTER     [_a-zA-Z]
IDCHAR     [_a-zA-Z0-9]
DEC_INT     0|{NZDIGIT}{DIGIT}*
OCT_INT     0[0-7]+
HEX_INT     0[xX][0-9a-fA-F]+
```

```

EXP          [eE][+-]?{DIGIT}+
FLOAT_BASE   ({DIGIT}+\.{DIGIT}+|{DIGIT}+\.\|\.{DIGIT}+)
FLOAT_LIT    ({FLOAT_BASE}{EXP}?|{DIGIT}+{EXP})
ID           {LETTER}{IDCHAR}*

```

这些定义分别用于识别十进制整数、八进制整数、十六进制整数、浮点数、指数形式浮点数和标识符。关键字规则放在标识符规则之前，保证`int`、`float`、`struct`、`return`、`if`、`else`、`while`被识别为关键字，而不是普通`ID`。

1.1.3.2 主要函数和宏的功能

`src/lexer.l` 中的主要辅助函数和宏如下：

- `lexical\_error(const char \*message)`：输出不带原始词素的 A 类错误，并递增 `lexical\_error\_count`。
- `lexical\_error\_text(const char \*message, const char \*text)`：输出带原始词素的 A 类错误，例如非法十六进制数、非法八进制数和未定义字符。
- `TOKEN(name)`：在独立 lexer 模式下返回 `TOK\_name`；后续接入 Bison 时可切换为返回 Bison token。
- `TOKEN\_VALUE(name)`：用于 `ID`、`TYPE`、`INT`、`FLOAT`、`RELOP` 等带属性值的 token。独立词法输出时由 `lexer\_main.c` 使用 `yytext` 打印词素。

`src/lexer\_main.c` 中的主要函数如下：

- `token\_name(TokenKind kind)`：将 token 枚举转换为输出名称，例如 `TOK\_INT` 转换为 `INT`。

- ``token_has_value(TokenKind kind)``：判断 token 是否需要额外打印词素值。

``INT``、``FLOAT``、``ID``、``TYPE``、``RELOP`` 会打印为 ``TOKEN: value``。

- ``main(int argc, char **argv)``：检查命令行参数，打开输入文件，调用 ``yyrestart()`` 初始化 Flex 输入流，再循环调用 ``yylex()``。默认只输出错误；设置 ``CMM_LEX_DUMP=1`` 后，在没有词法错误时打印完整 token 流。

1.1.3.3 注释和空白处理

空格、制表符、回车和换行不返回 token。单行注释使用 ``"/"[\n]*`` 直接跳过。

块注释使用 Flex start condition：

```
%x COMMENT
```

扫描到 ``/*`` 后记录起始行号并进入 ``COMMENT`` 状态；扫描到 ``*/`` 后回到初始状态。如果注释中再次出现 ``/*``，报告嵌套块注释错误；如果文件结束时仍在注释状态中，则使用记录的起始行号报告未闭合块注释错误。

1.1.3.4 错误处理

实践 1.1 中所有词法错误均输出为 A 类错误：

Error type A at Line 行号：错误说明。

当前实现覆盖以下情况：

- 未定义字符，例如 ``~``
- 数字开头的非法标识符

- 非法十六进制整数
- 非法八进制整数
- 非法普通浮点数
- 非法指数形式浮点数
- 嵌套块注释
- 未闭合块注释

错误规则放在合法数字规则之前，使非法字面量能整体识别并报告，而不是被拆成多个合法 token。

1.1.4 构建和运行

所有命令均在当前实践目录下执行：`cd submission/proj1`

检查环境：`make check-env`

构建词法分析器：`make lexer`

该命令生成：`build/cmm-lexer`

运行词法公开样例：`make lexer-demo`

手动查看某个源文件并运行词法分析：

```
nl -ba test/proj1-test1.cmm
CMM_LEX_DUMP=1 build/cmm-lexer test/proj1-test1.cmm
```

其中 ``nl -ba`` 用于打印带行号的源代码，方便对照输出；``CMM_LEX_DUMP=1`` 用于打开 token 流输出。

1.1.5 结果与结论分析

执行：`make lexer-demo`

当前会运行三个公开样例：

- ``test/proj1-2-3-6-exp1-1.cmm``：2.3.6 必做样例 1，输出未定义字符 ``~`` 的 A 类错误。
- ``test/proj1-2-3-7-exp1-0.cmm``：2.3.7 选做样例 1，输出八进制整数和十六进制整数 token。
- ``test/proj1-2-3-7-exp5-0.cmm``：2.3.7 选做样例 5，验证注释被跳过，只输出有效 token。

手写样例 ``test/proj1-test1.cmm`` 能正常输出 token 流，覆盖结构体、函数、变量定义、成员访问、赋值、函数调用、条件语句、``write`` 和 ``return`` 等常见词法单元。

本次实践完成了基于 Flex 的 C++ 词法分析器，实现了 token 识别、注释过滤、行号维护和 A 类错误报告。代码结构上将词法规则、token 定义和独立运行入口拆分到不同文件中，便于后续实践 1.2 在同一份 ``lexer.l`` 基础上接入 Bison。

1.2 Bison 语法分析

1.2.1 目的和任务

本次实践继续在 `submission/proj1` 目录中完成，在实践 1.1 词法分析器的基础上，使用 GNU Bison 实现 C++ 语言语法分析程序。程序需要判断输入源代码是否符合 C++ 语法规则；若没有词法或语法错误，则按照课程要求输出语法树；若存在错误，则输出 A 类或 B 类错误信息，并且不输出语法树。

本阶段主要任务如下：

1. 根据 `course/Appendix.pdf` 中的 C++ 文法编写 Bison 产生式。
2. 复用实践 1.1 的 `src/lexer.l`，使 Flex 能向 Bison 返回 token 和属性值。
3. 新增 AST 数据结构，保存语法单元、词法单元、行号、属性值和子节点关系。
4. 设置表达式优先级和结合性，处理赋值、逻辑、关系、算术、一元运算、数组访问、函数调用和结构体成员访问。
5. 对常见语法错误进行恢复，输出 B 类错误并尽量继续分析后续代码。
6. 编写 `Makefile` 和 `tools/run\_parser\_tests.sh`，使实践 1.2 可以在 `proj1` 目录中单独构建、单独演示。

1.2.2 项目结构和文件功能

实践 1.2 在实践 1.1 文件基础上新增语法分析和 AST 模块，当前目录结构如下：

```
submission/proj1/
├─ Makefile
├─ course/
│   └─ Appendix.pdf
│   └─ Project_1.pdf
├─ docs/
│   └─ practice1.2-parser.md
│   └─ practice1.2-parser-report.md
├─ src/
│   └─ ast.c
│   └─ ast.h
│   └─ lexer.l
│   └─ lexer_main.c
│   └─ parser.y
│   └─ parser_main.c
│   └─ token.h
├─ test/
│   └─ proj1-2-3-6-exp2-1.cmm
│   └─ proj1-2-3-6-exp3-1.cmm
│   └─ proj1-2-3-6-exp4-1.cmm
│   └─ proj1-test1.cmm
└─ tools/
    └─ check_env.sh
    └─ run_parser_tests.sh
```

各文件功能如下：

- ``Makefile``：实践一独立构建入口。``make parser`` 生成语法分析器和 AST 打印程序，``make parser-demo`` 运行语法公开样例。
- ``src/parser.y``：Bison 文法文件，是实践 1.2 的核心实现。它定义 token、非终结符类型、优先级、C-- 产生式、语法树构造动作和错误恢复动作。
- ``src/parser_main.c``：语法分析程序入口。它负责打开输入文件、调用 ``yyparse()``，并在词法错误数和语法错误数都为 0 时打印语法树。

- ``src/ast.h``: 声明 AST 节点结构和 AST 操作函数。
- ``src/ast.c``: 实现 AST 节点创建、子树合并、整数转换、先序打印和内存释放。
- ``src/lexer.l``: 继续复用实践 1.1 的词法规则。编译 parser 时通过 ``-DPARSER_BUILD`` 切换到 Bison 模式, 返回 ``parser.tab.h`` 中的 token, 并通过 ``yylval.node`` 传递终结符 AST 节点。
- ``src/lexer_main.c`` 和 ``src/token.h``: 保留实践 1.1 的独立词法分析入口和 token 枚举, 方便单独验证词法部分。
- ``tools/run_parser_tests.sh``: 运行 ``Project_1.pdf`` 中与语法树或语法错误相关的公开样例。
- ``test/proj1-2-3-6-exp2-1.cmm``: ``Project_1.pdf`` 2.3.6 必做样例 2, 检查语法错误输出。
- ``test/proj1-2-3-6-exp3-1.cmm``: ``Project_1.pdf`` 2.3.6 必做样例 3, 检查简单程序语法树。
- ``test/proj1-2-3-6-exp4-1.cmm``: ``Project_1.pdf`` 2.3.6 必做样例 4, 检查结构体相关语法树。
- ``test/proj1-test1.cmm``: 手写综合样例, 可用于现场展示 AST 输出。

1.2.3 在实践 1.1 基础上的修改和新增内容

实践 1.2 没有重写词法分析器, 而是在实践 1.1 的基础上做了以下扩展:

1. 新增 ``src/parser.y``, 用 Bison 描述 C-- 语法规则, 并在每个产生式动作中构造 AST 节点。
2. 新增 ``src/parser_main.c``, 作为语法分析阶段的命令行入口。
3. 新增 ``src/ast.h`` 和 ``src/ast.c``, 实现语法树数据结构和输出逻辑。
4. 修改 ``src/lexer.l``, 加入 ``PARSER_BUILD`` 条件编译逻辑。独立 lexer 模式仍返回 ``TOK_*``; parser 模式则包含 ``parser.tab.h``, 返回 Bison token。

5. 修改 token 返回动作。parser 模式下, `TOKEN(name)` 和 `TOKEN_VALUE(name)` 会创建终结符 AST 节点并写入 `yyval.node`, 从而把 `ID`、`TYPE`、`INT`、`FLOAT`、`RELOP` 等属性传给 Bison。
6. 修改 `Makefile`, 新增 `parser` 和 `parser-demo` 目标。`make parser` 先运行 Bison 生成 `build/parser.tab.c` 和 `build/parser.tab.h`, 再用 `flex -DPARSER_BUILD` 生成 parser 专用 lexer, 最后链接生成 `build/cmmc`。
7. 新增 `tools/run_parser_tests.sh`, 用于在当前目录运行公开语法样例。

1.2.4 主要函数和模块说明

1.2.4.1 `parser.y` 中的辅助函数

- `node_line(AstNode *node)`: 取得 AST 节点行号; 节点为空时使用当前 `yylineno`, 用于空产生式或错误恢复场景。
- `syntax_error_at(int line, const char *msg)`: 输出 B 类语法错误, 并使用 `last_syntax_error_line` 避免同一行重复报错。若已经存在词法错误, 则不再输出语法错误, 避免混淆错误类型。
- `ast_has_array_vardec(AstNode *node)`: 递归检查函数声明参数中是否出现数组形式的变量声明, 用于报告课程样例中需要识别的语法问题。
- `yyerror(const char *msg)`: Bison 默认错误回调函数。它将默认语法错误统一输出为 `Error type B at Line 行号: Syntax error.`。

1.2.4.2 `ast.h` / `ast.c` 中的函数

- `ast_token(const char *name, const char *value, int line)`: 创建终结符节点, 保存 token 名称、可选属性值和行号。
- `ast_node(const char *name, int line, size_t child_count, ...)`: 创建非终结符节点, 并接收可变数量的子节点。
- `ast_adopt(const char *name, int line, AstNode *first, AstNode *second)`: 合并两个节点的子节点, 适合需要重新组织子树的场景。
- `ast_parse_int(const char *text)`: 使用 `strtol(text, NULL, 0)` 将十进制、八进制、十六进制整数字符串转换为十进制数值。
- `ast_print(const AstNode *node, int indent)`: 按照课程要求先序遍历输出语法树。非终结符打印 `Name (line)`; `ID`、`TYPE`、`INT`、`FLOAT` 打印属性值; 子节点缩进 2 个空格。
- `ast_free(AstNode *node)`: 递归释放 AST 节点、名称字符串、属性字符串和子节点数组。

1.2.4.3 `parser_main.c` 中的入口逻辑

`main()` 的执行流程如下:

1. 检查命令行参数, 要求输入一个 `.cmm` 文件。
2. 打开源文件, 并调用 `yyrestart(yyin)` 设置 Flex 输入流。
3. 调用 `yyparse()` 执行语法分析。
4. 如果 `lexical_error_count == 0`、`syntax_error_count == 0` 且 `yyparse()` 返回 0, 则调用

``ast_print(parse_root, 0)`` 输出语法树。

5. 调用 ``ast_free(parse_root)`` 释放内存，关闭输入文件。

这种设计保证错误样例只输出错误信息，不会混入语法树内容。

1.2.5 语法规则和错误恢复

``src/parser.y`` 覆盖了 C-- 的主要语法单元：

- 外部定义：`Program`、`ExtDefList`、`ExtDef`、`ExtDecList`
- 类型说明：`Specifier`、`StructSpecifier`、`OptTag`、`Tag`
- 变量和函数：`VarDec`、`FunDec`、`VarList`、`ParamDec`
- 复合语句和语句：`CompSt`、`StmtList`、`Stmt`
- 局部定义：`DefList`、`Def`、`DecList`、`Dec`
- 表达式和实参：`Exp`、`Args`

表达式优先级从低到高设置为：

ASSIGNOP

OR

AND

RELOP

PLUS / MINUS

STAR / DIV

NOT / UMINUS

LP / RP / LB / RB / DOT

同时使用 ``LOWER_THAN_ELSE`` 和 ``ELSE`` 处理悬空 ``else``，使 ``else`` 与最近的 ``if`` 匹配。

错误恢复方面，当前实现针对常见错误添加了局部恢复产生式，例如数组维度不是整数、函数参数尾逗号、函数调用实参尾逗号、全局变量初始化、语句后出现变量定义、空 `while` 语句体、数组初始化列表、结构体成员访问缺少成员名等。这样可以在报告错误后继续分析后续输入，减少连锁错误。

1.2.6 构建和运行

所有命令均在当前实践目录下执行：`cd submission/proj1`

构建语法分析器：`make parser`

该命令生成：`build/cmmc`

运行语法公开样例：`make parser-demo`

手动查看某个源文件并运行语法分析：

```
nl -ba test/proj1-test1.cmm  
build/cmmc test/proj1-test1.cmm
```

1.2.7 结果与结论分析

执行：`make parser-demo`

当前会运行三个公开样例：

- ``test/proj1-2-3-6-exp2-1.cmm``：2.3.6 必做样例 2，输出两处 B 类语法错误。
- ``test/proj1-2-3-6-exp3-1.cmm``：2.3.6 必做样例 3，输出简单函数的语法树。

· ``test/proj1-2-3-6-exp4-1.cmm``: 2.3.6 必做样例 4, 输出结构体声明和结构体变量访问相关语法树。

手写样例 ``test/proj1-test1.cmm`` 可以正常输出语法树, 覆盖结构体定义、函数定义、局部变量定义、成员访问、函数调用、``if-else``、``write`` 和 ``return`` 等语法结构。

本次实践完成了基于 Bison 的 C++ 语法分析器, 与实践 1.1 的 Flex 词法分析器完成衔接。

程序在正确输入上能够按照课程格式输出语法树, 在错误输入上能够输出 A 类或 B 类错误信息。新增的 AST 模块和 parser 入口也为后续语义分析阶段提供了清晰的数据结构基础。

2 编译原理课程设计报告：实践 2 语义分析

课程设计名称：编译原理课程设计

实践内容：实践 2 语义分析

2.1 目的和任务

本次实践在 `submission/proj2` 目录中独立完成，目标是在实践 1 词法分析、语法分析和 AST 构造的基础上，实现 C-- 语言的语义分析程序。程序在语法分析成功后遍历语法树，检查变量、函数、数组、结构体和表达式的语义约束；若发现语义错误，则按照课程要求输出错误类型、行号和说明文字；若没有语义错误，则不输出内容。

本阶段主要任务如下：

1. 建立 C-- 语言类型系统，支持 `int`、`float`、数组、结构体和函数类型。
2. 建立符号表和作用域结构，记录变量、函数、结构体定义及其可见范围。
3. 检查未定义变量、未定义函数、变量重定义、函数重定义、结构体重名、未定义结构体等错误。
4. 检查赋值、运算、返回语句、函数调用、数组访问和结构体成员访问的类型一致性。
5. 支持函数声明与函数定义，并检查函数未定义和声明不一致问题。
6. 保留实践 1 的词法、语法和 AST 前端，使语义分析可以直接复用已有语法树。
7. 编写 `Makefile` 和 `tools/run\_semantic\_tests.sh`，使实践二可以在 `proj2` 目录中单独构建、

单独演示。

2.2 项目结构和文件功能

实践二相关文件均位于 `submission/proj2` 下，目录结构如下：

```
submission/proj2/
├── Makefile
├── course/
│   ├── Appendix.pdf
│   └── Project_2.pdf
├── docs/
│   ├── practice2-semantic.md
│   ├── practice2-semantic-report.md
│   └── practice2-semantic-report.docx
├── src/
│   ├── ast.c
│   ├── ast.h
│   ├── lexer.l
│   ├── lexer_main.c
│   ├── parser.y
│   ├── parser_main.c
│   ├── semantic.c
│   ├── semantic.h
│   └── token.h
├── test/
│   ├── proj2-3-3-6-exp1-1.cmm
│   ├── proj2-3-3-6-exp2-1.cmm
│   ├── proj2-3-3-6-exp3-1.cmm
│   ├── proj2-3-3-6-exp5-1.cmm
│   ├── proj2-3-3-6-exp8-1.cmm
│   ├── proj2-3-3-6-exp14-1.cmm
│   ├── proj2-3-3-6-exp17-1.cmm
│   ├── proj2-3-3-7-exp1-0.cmm
│   ├── proj2-3-3-7-exp2-0.cmm
│   ├── proj2-3-3-7-exp3-0.cmm
│   ├── proj2-3-3-7-exp4-0.cmm
│   └── proj2-test1.cmm
```

```
└─ tools/
   └─ check_env.sh
   └─ run_semantic_tests.sh
```

各文件功能如下：

- ``Makefile``：实践二独立构建入口。``make semantic`` 构建语义分析器，``make semantic-demo`` 运行公开样例，``make test`` 作为演示别名。
- ``src/semantic.c``：实践二核心实现，包含类型系统、字段链表、符号表、作用域、函数表、结构体表和语义遍历逻辑。
- ``src/semantic.h``：声明语义分析入口 ``semantic_check(AstNode *root)``。
- ``src/parser_main.c``：主程序入口。默认行为保持实践 1.2 的 AST 输出；设置 ``CMM_SEMANTIC=1`` 时调用 ``semantic_check()`` 执行语义分析。
- ``src/parser.y``：Bison 文法和 AST 构造动作。实践二继续使用该文件生成语法树，语义分析基于语法树进行。
- ``src/lexer.l``：Flex 词法规则。实践二继续复用实践一的词法分析器。
- ``src/ast.c``、``src/ast.h``：AST 节点结构、节点创建、整数解析、先序打印和内存释放。
- ``src/lexer_main.c``、``src/token.h``：保留独立词法分析入口和 token 枚举，方便单独验证前端词法部分。
- ``tools/run_semantic_tests.sh``：运行 ``Project_2.pdf`` 中 3.3.6 和 3.3.7 的公开样例，并打印源文件路径和语义分析输出。
- ``test/``：按真实章节号命名的公开样例，以及手写综合样例 ``proj2-test1.cmm``。

2.3 在实践 1 基础上的修改和新增内容

实践二是在实践一前端的基础上扩展语义检查，没有重写词法和语法分析部分。主要新增和修改如下：

1. 新增 `src/semantic.h`，声明语义分析统一入口。
2. 新增 `src/semantic.c`，实现类型系统、符号表、函数表、结构体表、作用域管理和 AST 语义遍历。
3. 修改 `src/parser\_main.c`，增加 `CMM\_SEMANTIC=1` 模式。未设置该环境变量时仍输出 AST；设置后只做语义检查，正确程序不输出内容。
4. 修改 `Makefile`，收敛为实践二当前目录可独立运行的目标，只保留 lexer、parser、semantic 和演示相关入口。
5. 新增 `tools/run\_semantic\_tests.sh`，使用当前 `test/` 目录中的公开样例进行演示。
6. 新增 `test/proj2-3-3-6-...` 和 `test/proj2-3-3-7-...` 样例文件，文件名中的 `3-3-6` 和 `3-3-7` 分别对应 `Project\_2.pdf` 的必做样例和选做样例小节。
7. 对表达式赋值检查做了级联错误控制：当赋值左右任一侧已经是错误类型时，不继续报告左值错误或类型不匹配错误，避免一个根因引出多个无关错误。

2.4 主要数据结构和函数功能

2.4.1 类型系统

`src/semantic.c` 中使用 `TypeKind` 和 `Type` 表示 C-- 类型：

TY_ERROR	错误类型，用于抑制级联误报
TY_INT	int 基本类型

TY_FLOAT float 基本类型
TY_ARRAY 数组类型
TY_STRUCT 结构体类型
TY_FUNC 函数类型

`Type` 结构中保存数组元素类型和维度、结构体名和字段链表、函数返回类型、形参链表、形参数量、函数是否已定义以及声明行号。

相关函数如下：

- `new\_type(TypeKind kind)`：创建指定种类的类型对象。
- `type\_int()`、`type\_float()`、`type\_error()`：返回基本类型和错误类型的共享对象。
- `array\_of(Type \*elem, int size)`：构造数组类型。
- `struct\_type(const char \*name, Field \*fields)`：构造结构体类型。
- `func\_type(Type \*ret, Field \*params, int param\_count, int defined, int line)`：构造函数类型。
- `type\_equal(const Type \*a, const Type \*b)`：判断两个类型是否等价。基本类型要求种类一致，数组要求元素类型和维度一致，具名结构体按结构体名比较。
- `is\_numeric(const Type \*type)`：判断类型是否可参与数值运算。

2.4.2 字段、符号和作用域

语义分析使用三类链表结构：

- `Field`：表示结构体字段或函数参数，保存名称、类型、行号和下一个字段。
- `Symbol`：表示变量或函数符号，保存名称、类型、定义行和下一个符号。
- `Scope`：表示一个作用域，保存当前作用域符号链表和父作用域指针。

相关函数如下：

- ``push_scope()``: 进入新作用域。
- ``pop_scope()``: 退出当前作用域。
- ``find_in_scope(Scope *scope, const char *name)``: 只在指定作用域查找符号。
- ``find_symbol(const char *name)``: 从当前作用域向外逐层查找变量符号。
- ``add_symbol_current(const char *name, Type *type, int line)``: 向当前作用域加入变量, 并检查同作用域重定义。
- ``find_function(const char *name)``: 在全局函数表中查找函数。
- ``find_struct_def(const char *name)``: 在结构体表中查找结构体定义。
- ``add_struct_def(const char *name, Type *type, int line)``: 加入结构体定义, 并检查结构体名重复。

2.4.3 AST 辅助函数

语义分析需要频繁读取 AST 节点, 主要辅助函数如下:

- ``is_node(const AstNode *node, const char *name)``: 判断节点名称。
- ``child(const AstNode *node, size_t idx)``: 获取指定子节点。
- ``has_child_name(const AstNode *node, const char *name)``: 判断是否存在指定名称子节点。
- ``first_child_name(const AstNode *node, const char *name)``: 返回第一个指定名称子节点。
- ``read_vardec(AstNode *var_dec, VarInfo *out)``: 解析变量声明节点, 提取变量名、行号和数组维度。

- ``apply_dims(Type *base, const VarInfo *info)``: 把数组维度应用到基础类型上, 得到完整变量类型。

2.4.4 语义检查主流程函数

- ``semantic_check(AstNode *root)``: 语义分析总入口。它初始化全局状态, 创建全局作用域, 加入内置函数 ``read`` 和 ``write``, 遍历外部定义, 最后检查只声明未定义的函数。

- ``analyze_ext_def_list(AstNode *node)``: 遍历外部定义列表。

- ``analyze_ext_def(AstNode *node)``: 处理全局变量、结构体定义、函数声明和函数定义。

- ``handle_function(AstNode *ext_def, Type *ret, int is_definition)``: 处理函数声明或定义, 检查函数重定义、声明不一致, 并在函数定义中创建形参作用域。

- ``analyze_compst(AstNode *node, Type *return_type, int create_scope)``: 分析复合语句, 进入或复用作用域, 依次检查局部定义和语句列表。

- ``analyze_def_list(AstNode *node, int as_fields, Field **fields)``: 分析局部变量定义或结构体字段定义。

- ``collect_dec()``、``collect_dec_list()``: 收集变量或字段声明, 并检查字段初始化、字段重定义和局部变量初始化类型。

- ``analyze_stmt_list()``、``analyze_stmt()``: 分析语句列表、表达式语句、复合语句、返回语句、条件语句和循环语句。

- ``analyze_exp(AstNode *node, int *is_lvalue)``: 表达式类型综合函数。它检查标识符、常量、

赋值、二元运算、函数调用、数组访问、结构体成员访问等表达式，并返回表达式类型。

- ``check_undefined_functions()``: 语义遍历结束后检查只有声明没有定义的函数。

2.4.5 错误输出函数

- ``report_error(int type, int line, const char *msg)``: 直接输出语义错误并累加错误数。
- ``queue_error(int type, int line, const char *msg)``: 暂存需要延后输出的错误，主要用于函数声明不一致。
- ``flush_pending_errors()``: 语义分析结束时输出暂存错误。

错误输出格式为:

Error type 错误类型 at Line 行号: 错误说明.

2.5 语义检查内容

当前实现覆盖以下语义检查:

- 类型 1: 变量未定义。
- 类型 2: 函数未定义。
- 类型 3: 变量重定义。
- 类型 4: 函数重定义。
- 类型 5: 赋值号两侧类型不匹配。
- 类型 6: 赋值号左侧不是合法左值。

- 类型 7: 运算符两侧操作数类型不匹配。
- 类型 8: 返回值类型与函数返回类型不匹配。
- 类型 9: 函数调用实参与形参数量或类型不匹配。
- 类型 10: 对非数组变量使用下标。
- 类型 11: 对非函数变量使用函数调用。
- 类型 12: 数组下标不是整数。
- 类型 13: 对非结构体变量使用成员访问。
- 类型 14: 访问结构体中不存在的字段。
- 类型 15: 结构体字段重复定义或字段初始化。
- 类型 16: 结构体名称重复。
- 类型 17: 使用未定义结构体。
- 类型 18: 函数声明后未定义。
- 类型 19: 函数声明不一致。

此外，程序支持嵌套作用域。默认情况下，内层作用域可以遮蔽外层变量；若设置`CMM\_NO\_SCOPE=1`，则启用旧口径的外层同名冲突检查。

2.6 构建和运行

所有命令均在当前实践目录下执行：`cd submission/proj2`

检查环境：`make check-env`

构建语义分析器：`make semantic`

该命令生成：build/cmmc

运行公开样例演示：make semantic-demo

也可以执行：make semantic-test

手动查看某个源文件并运行语义分析：

```
nl -ba test/proj2-test1.cmm
```

```
CMM_SEMANTIC=1 build/cmmc test/proj2-test1.cmm
```

其中 `nl -ba` 用于打印带行号的源代码，方便对照错误行号；`CMM\_SEMANTIC=1` 用于打开语义分析模式。

2.7 结果与结论分析

执行：make semantic-demo

当前会运行 `Project\_2.pdf` 中的公开样例：

- `test/proj2-3-3-6-exp1-1.cmm`：3.3.6 必做样例 1，输出未定义变量错误。
- `test/proj2-3-3-6-exp2-1.cmm`：3.3.6 必做样例 2，输出未定义函数错误。
- `test/proj2-3-3-6-exp3-1.cmm`：3.3.6 必做样例 3，输出变量重定义错误。
- `test/proj2-3-3-6-exp5-1.cmm`：3.3.6 必做样例 5，输出赋值类型不匹配错误。
- `test/proj2-3-3-6-exp8-1.cmm`：3.3.6 必做样例 8，输出返回类型不匹配错误。
- `test/proj2-3-3-6-exp14-1.cmm`：3.3.6 必做样例 14，输出结构体字段不存在错误。
- `test/proj2-3-3-6-exp17-1.cmm`：3.3.6 必做样例 17，输出未定义结构体错误。
- `test/proj2-3-3-7-exp1-0.cmm`：3.3.7 选做样例 1，函数声明与定义一致，不输出错误。

- ``test/proj2-3-3-7-exp2-0.cmm``: 3.3.7 选做样例 2, 输出函数未定义和声明不一致错误。
- ``test/proj2-3-3-7-exp3-0.cmm``: 3.3.7 选做样例 3, 嵌套作用域下不输出错误。
- ``test/proj2-3-3-7-exp4-0.cmm``: 3.3.7 选做样例 4, 输出同一作用域变量重定义错误。

手写样例 ``test/proj2-test1.cmm`` 覆盖结构体定义、函数参数、结构体成员访问、赋值类型不匹配、函数实参不匹配和未定义变量等情况, 可用于现场展示综合语义检查结果。

本次实践完成了 C-- 语言语义分析器。程序能够在语法分析成功后检查变量、函数、数组、结构体和表达式相关语义错误, 并按课程要求输出错误类型和行号。当前目录可以独立完成构建与演示, 后续实践可继续在 AST 和语义检查结果基础上进行中间代码生成。

3 编译原理课程设计报告：实践 3 中间代码生成

课程设计名称：编译原理课程设计

实践内容：实践 3 中间代码生成

3.1 目的和任务

本次实践在 `submission/proj3` 目录中独立完成，目标是在词法分析、语法分析和语义分析完成的基础上，将 C++ 源程序翻译为线性中间代码。中间代码采用三地址代码形式，用于表示赋值、算术运算、条件跳转、循环、函数调用、参数传递、数组访问、结构体字段访问和输入输出操作。

本阶段主要任务如下：

1. 在语义检查通过后遍历 AST，生成 `.ir` 中间代码文件。
2. 生成临时变量和跳转标号，翻译表达式、语句和控制流。
3. 翻译函数定义、形参、实参、函数调用和返回语句。
4. 翻译数组和结构体变量的空间申请、地址计算和间接读写。
5. 特殊处理内置函数 `read` 和 `write`，生成 `READ`、`WRITE` 指令。
6. 保留实践 1 和实践 2 的前端与语义检查，使错误程序不会进入 IR 生成阶段。
7. 编写 `Makefile` 和 `tools/run\_ir\_tests.sh`，使实践三可以在 `proj3` 目录中单独构建、单独演示。

3.2 项目结构和文件功能

实践三相关文件均位于 `submission/proj3` 下，目录结构如下：

```
submission/proj3/
├── Makefile
├── course/
│   ├── Appendix.pdf
│   ├── Project_3.pdf
│   ├── Tests_3.zip
│   └── irsim.zip
├── docs/
│   ├── cmm-grammar.md
│   ├── practice3-ir.md
│   ├── practice3-ir-report.md
│   └── practice3-ir-report.docx
├── src/
│   ├── ast.c
│   ├── ast.h
│   ├── irgen.c
│   ├── irgen.h
│   ├── lexer.l
│   ├── lexer_main.c
│   ├── parser.y
│   ├── parser_main.c
│   ├── semantic.c
│   ├── semantic.h
│   └── token.h
├── test/
│   ├── proj3-4-3-6-exp1-1.cmm
│   ├── proj3-4-3-6-exp2-1.cmm
│   ├── proj3-4-3-7-exp1-0.cmm
│   ├── proj3-4-3-7-exp2-0.cmm
│   └── proj3-test1.cmm
└── tools/
    ├── build_irsim_image.sh
    ├── check_env.sh
    ├── run_ir_tests.sh
    └── run_irsim_tests.sh
```

└─ run_one_irsim_case.sh

各文件功能如下：

- ``Makefile``：实践三独立构建入口。``make ir`` 构建 IR 生成器，``make ir-demo`` 运行公开样例并打印生成的 IR。
- ``src/irgen.c``：实践三核心实现，包含 IR 类型系统、局部符号表、结构体字段偏移、函数签名收集、表达式翻译、语句翻译和文件输出。
- ``src/irgen.h``：声明 IR 生成入口 ``ir_generate(AstNode *root, const char *output_path)``。
- ``src/parser_main.c``：主程序入口。默认输出 AST；设置 ``CMM_SEMANTIC=1`` 时只做语义检查；设置 ``CMM_IR=1`` 时先做语义检查，再调用 ``ir_generate()`` 输出 IR。
- ``src/semantic.c``、``src/semantic.h``：复用实践二语义检查，保证只有语义正确的程序进入 IR 翻译。
- ``src/parser.y``、``src/lexer.l``：复用前端，负责生成 IR 翻译所需 AST。
- ``src/ast.c``、``src/ast.h``：AST 节点结构、节点创建、整数解析、先序打印和内存释放。
- ``tools/run_ir_tests.sh``：运行 ``Project_3.pdf`` 中 4.3.6 和 4.3.7 的公开样例，生成 IR 文件并打印前 120 行。
- ``tools/build_irsim_image.sh``：构建 Python 3.8 + PyQt5 的 IRSim Docker 镜像。
- ``tools/run_one_irsim_case.sh``：生成并运行单个当前样例的 IR，打印 IRSim 控制台输出。
- ``tools/run_irsim_tests.sh``：批量生成并运行当前样例 IR，检查 IRSim 输出是否符合预期。
- ``test/``：按真实章节号命名的公开样例，以及手写综合样例 ``proj3-test1.cmm``。

3.3 在前两次实践基础上的修改和新增内容

实践三是在实践一、实践二基础上的后续扩展，主要新增和修改如下：

1. 新增 `src/irgen.h`，声明中间代码生成入口。
2. 新增 `src/irgen.c`，实现 AST 到三地址中间代码的翻译。
3. 修改 `src/parser\_main.c`，增加 `CMM\_IR=1` 模式。该模式要求命令行传入输入 `.cmm` 文件和输出 `.ir` 文件。
4. 修改 `Makefile`，收敛为实践三当前目录可独立运行的目标，只保留 lexer、parser、semantic、ir 和演示相关入口。
5. 新增 `tools/run\_ir\_tests.sh`，使用当前 `test/` 目录中的公开样例生成并打印 IR。
6. 新增 `test/proj3-4-3-6-...` 和 `test/proj3-4-3-7-...` 样例文件，文件名中的 `4-3-6` 和 `4-3-7` 分别对应 `Project\_3.pdf` 的必做样例和选做样例小节。
7. 新增 IRSim 单样例和批量验证脚本，使生成的 IR 可以直接用课程模拟器运行。
8. 删除与当前目录演示无关的旧批量测试和报告目录，使实践三目录只保留当前演示所需材料。

3.4 主要数据结构和函数功能

3.4.1 IR 类型系统

`src/irgen.c` 中使用 `IrTypeKind` 和 `IrType` 描述生成 IR 时需要的类型信息：

IR_TY_INT	int 基本类型
IR_TY_FLOAT	float 基本类型
IR_TY_ARRAY	数组类型
IR_TY_STRUCT	结构体类型

相关函数如下：

- ``new_type(IrTypeKind kind)``：创建指定种类的 IR 类型。
- ``type_int()``、``type_float()``：返回基本类型对象。
- ``array_of(IrType *elem, int size)``：构造数组类型。
- ``struct_type(const char *name, IrField *fields)``：构造结构体类型。
- ``type_size(IrType *type)``：计算类型占用字节数。基本类型为 4 字节；数组大小为元素大小乘以长度；结构体大小为字段大小之和。

小乘以长度；结构体大小为字段大小之和。

3.4.2 字段、符号、作用域和函数签名

IR 生成阶段维护轻量级符号信息：

- ``IrField``：表示结构体字段或函数参数，保存字段名、类型、偏移和下一个字段。
- ``IrSymbol``：表示局部变量或形参，保存名称、类型以及是否为地址参数。
- ``IrScope``：表示嵌套作用域，保存当前作用域变量链表和父作用域。
- ``IrFunc``：表示函数签名，保存函数名、返回类型和参数链表。

相关函数如下：

- ``push_scope()``、``pop_scope()``：进入和退出作用域。
- ``find_symbol(const char *name)``：从当前作用域向外查找变量。
- ``add_symbol(const char *name, IrType *type, int is_addr_param)``：向当前作用域加入变量或

形参。

- ``find_func(const char *name)``: 查找函数签名。
- ``add_func(const char *name, IrType *ret, IrField *params)``: 记录函数签名。
- ``find_struct_def(const char *name)``: 查找结构体定义。
- ``find_field(IrField *fields, const char *name)``: 查找结构体字段。

3.4.3 输出和临时对象生成

- ``emit(const char *fmt, ...)``: 向输出文件写入一条 IR 指令。
- ``new_temp()``: 生成临时变量名, 例如 ``t1``、``t2``。
- ``new_label()``: 生成跳转标号, 例如 ``label1``、``label2``。
- ``materialize_addr(char *addr)``: 将 ``&x`` 形式的地址表达式必要时保存到临时变量, 避免间接读写时生成不稳定格式。

3.4.4 声明、类型和函数签名收集

- ``read_vardec(AstNode *var_dec, VarInfo *out_info)``: 从变量声明 AST 中提取变量名和数组维度。
- ``apply_dims(IrType *base, const VarInfo *info)``: 把数组维度应用到基础类型, 得到完整变量类型。
- ``analyze_specifier(AstNode *specifier)``: 分析类型说明节点, 返回对应 IR 类型。

- ``analyze_struct_specifier(AstNode *node)``: 分析结构体定义或结构体引用, 并记录字段偏移。
- ``collect_def_list()``、``collect_dec_list()``、``collect_dec()``: 收集局部变量或结构体字段声明; 对数组和结构体局部变量生成 ``DEC`` 指令。
- ``collect_params(AstNode *var_list)``: 收集函数参数类型。
- ``collect_function_sigs(AstNode *node)``: 翻译函数体前先遍历全局定义, 记录所有函数签名, 便于后续函数调用时判断参数传值或传地址。

3.4.5 语句和表达式翻译

- ``gen_ext_def_list()``、``gen_ext_def()``: 翻译外部定义。函数定义会输出 ``FUNCTION f:``, 再翻译形参和函数体。
- ``gen_params(IrField *params)``: 输出 ``PARAM`` 指令, 并将形参加入当前作用域。
- ``gen_compst(AstNode *node, int create_scope)``: 翻译复合语句, 处理局部定义和语句列表。
- ``gen_stmt_list()``、``gen_stmt()``: 翻译表达式语句、复合语句、返回语句、条件语句和循环语句。
- ``gen_exp(AstNode *node)``: 翻译普通表达式, 返回保存结果的变量名、临时变量或立即数。
- ``gen_addr(AstNode *node, IrType **out_type)``: 翻译左值地址, 处理变量、数组元素和结构体字段。

- ``gen_cond(AstNode *node, const char *true_label, const char *false_label)``：翻译布尔表达式和条件跳转。
- ``gen_call(AstNode *node)``：翻译函数调用。普通函数生成 ``ARG`` 和 ``CALL``；内置 ``read`` 生成 ``READ``，内置 ``write`` 生成 ``WRITE``。
- ``collect_args()``、``emit_args_reverse()``：收集并逆序输出实参，满足课程 IR 中函数调用参数传递格式。

3.4.6 IR 生成总入口

``ir_generate(AstNode *root, const char *output_path)`` 是实践三的核心入口，执行流程如下：

1. 打开输出 ``.ir`` 文件。
2. 初始化临时变量编号、标号编号、作用域、结构体表和函数表。
3. 创建全局作用域。
4. 遍历 AST 收集函数签名。
5. 遍历 AST 生成所有函数和语句的 IR。
6. 关闭输出文件并返回状态码。

3.5 IR 指令格式

当前实现支持的主要三地址指令如下：

```
FUNCTION f :
PARAM x
LABEL label :
```

```
x := y
x := y + z
x := y - z
x := y * z
x := y / z
x := &y
x := *y
*x := y
GOTO label
IF x relop y GOTO label
ARG x
x := CALL f
READ x
WRITE x
RETURN x
DEC x size
```

数组和结构体通过地址计算实现。例如数组元素访问会先计算下标偏移，再使用 `&a + offset` 得到地址；结构体成员访问通过字段偏移定位成员地址。数组和结构体作为函数参数时按地址传递，普通整型参数按值传递。

3.6 构建和运行

所有命令均在当前实践目录下执行：`cd submission/proj3`

检查环境：`make check-env`

构建 IR 生成器：`make ir`

该命令生成：`build/cmmc`

运行公开样例演示：`make ir-demo`

也可以执行：`make ir-test`

手动查看某个源文件并生成 IR：

```
nl -ba test/proj3-test1.cmm
CMM_IR=1 build/cmmc test/proj3-test1.cmm /tmp/proj3-test1.ir
sed -n '1,120p' /tmp/proj3-test1.ir
```

使用 IRSim 批量验证: `make irsim-test`

使用 IRSim 单独验证手写样例:

```
./tools/run_one_irsim_case.sh proj3-test1
```

如果本机缺少 IRSim Docker 镜像, 可先执行:

```
./tools/build_irsim_image.sh
```

3.7 结果与结论分析

执行: `make ir-demo`

当前会运行 `Project\_3.pdf` 中的公开样例:

- `test/proj3-4-3-6-exp1-1.cmm`: 4.3.6 必做样例 1, 生成符号函数相关 IR。
- `test/proj3-4-3-6-exp2-1.cmm`: 4.3.6 必做样例 2, 生成递归阶乘相关 IR。
- `test/proj3-4-3-7-exp1-0.cmm`: 4.3.7 选做样例 1, 生成结构体变量和结构体参数相关 IR。
- `test/proj3-4-3-7-exp2-0.cmm`: 4.3.7 选做样例 2, 生成一维数组参数和二维数组相关 IR。
- `test/proj3-test1.cmm`: 手写样例, 生成循环、函数调用和 `write` 相关 IR。

生成结果中可以看到 `FUNCTION`、`PARAM`、`LABEL`、`IF ... GOTO`、`ARG`、`CALL`、`READ`、`WRITE`、`RETURN`、`DEC` 等指令, 说明函数、控制流、调用、数组和结构体相关翻译均已覆盖。

执行: `make irsim-test`

当前 IRSim 验证结果为: `irsim tests: 5 passed, 0 failed`

本次实践完成了 C-- 到三地址中间代码的生成。程序能够在语义检查成功后输出符合课程格式的线性 IR，并支持必做内容以及结构体变量、结构体参数、一维数组参数和高维数组等选做内容。当前目录可以独立完成构建与演示，生成的 IR 可作为后续目标代码生成和优化实践的输入。

4 编译原理课程设计报告：实践 4 目标代码生成

课程设计名称：编译原理课程设计

实践内容：实践 4 目标代码生成

4.1 目的和任务

本次实践在 `submission/proj4` 目录中独立完成，目标是在已有词法分析、语法分析、语义分析和中间代码生成的基础上，将 C-- 程序翻译为可在 SPIM Simulator 中运行的 MIPS32 汇编代码。

本阶段主要任务如下：

1. 增加目标代码生成入口，接收 C-- 输入文件和 `.s` 输出文件。
2. 复用实践三生成的线性 IR，完成从 IR 到 MIPS32 指令的翻译。
3. 实现函数栈帧、参数传递、返回值、递归调用和局部数据存储。
4. 翻译赋值、算术运算、条件跳转、循环、数组地址计算和间接访存。
5. 使用 SPIM 对生成的汇编程序进行运行验证。
6. 编写 `Makefile` 和 `tools/run\_mips\_tests.sh`，使实践四可以在 `proj4` 目录中单独构建、单独演示。

4.2 项目结构和文件功能

实践四相关文件均位于 `submission/proj4` 下，目录结构如下：

```
submission/proj4/
├─ Makefile
├─ course/
│   ├── Appendix.pdf
│   ├── MIPS32_and_SPIM.pdf
│   └─ Project_4.pdf
├─ docs/
│   ├── cmm-grammar.md
│   ├── practice4-mips.md
│   ├── practice4-mips-report.md
│   └─ practice4-mips-report.docx
├─ src/
│   ├── ast.c
│   ├── ast.h
│   ├── irgen.c
│   ├── irgen.h
│   ├── lexer.l
│   ├── lexer_main.c
│   ├── mipsgen.c
│   ├── mipsgen.h
│   ├── parser.y
│   ├── parser_main.c
│   ├── semantic.c
│   ├── semantic.h
│   └─ token.h
├─ test/
│   ├── proj4-5-3-6-expl-1.cmm
│   ├── proj4-5-3-6-exp2-1.cmm
│   └─ proj4-test1.cmm
└─ tools/
    ├── check_env.sh
    └─ run_mips_tests.sh
```

各文件功能如下：

- ``Makefile``：实践四独立构建入口。``make mips`` 构建 MIPS 生成器，``make mips-demo`` 生成汇编并运行 SPIM。
- ``src/mipsgen.c``：实践四核心实现，读取线性 IR，按函数分组，计算栈帧布局，并输出

MIPS32 汇编。

- ``src/mipsgen.h``：声明后端入口 ``mips_generate_from_ir(const char *ir_path, const char *output_path)``。
- ``src/parser_main.c``：主程序入口。一参数时输出 AST；设置 ``CMM_SEMANTIC=1`` 时只做语义检查；设置 ``CMM_IR=1`` 时生成 IR；两参数模式下生成 MIPS 汇编。
- ``src/irgen.c``、``src/irgen.h``：复用实践三，将 AST 翻译为临时 IR。
- ``src/semantic.c``、``src/semantic.h``：复用实践二，在生成 IR 和 MIPS 前做语义检查。
- ``src/parser.y``、``src/lexer.l``：复用前端，负责生成后端所需 AST。
- ``src/ast.c``、``src/ast.h``：AST 节点结构、节点创建、整数解析、先序打印和内存释放。
- ``tools/run_mips_tests.sh``：运行 ``Project_4.pdf`` 中 5.3.6 的公开样例和手写样例，打印汇编文件前 120 行，并调用 SPIM 输出运行结果。
- ``test/``：按真实章节号命名的公开样例，以及手写综合样例 ``proj4-test1.cmm``。

4.3 在前三次实践基础上的修改和新增内容

实践四是在实践三 IR 生成器基础上的后端扩展，主要新增和修改如下：

1. 新增 ``src/mipsgen.h``，声明 MIPS 目标代码生成入口。
2. 新增 ``src/mipsgen.c``，实现 IR 到 MIPS32 汇编的翻译。
3. 修改 ``src/parser_main.c``，支持两参数目标代码生成模式：``build/cmmc input.cmm output.s``。
4. 修改 ``Makefile``，收敛为实践四当前目录可独立运行的目标，只保留

lexer、parser、semantic、ir、mips 和演示相关入口。

5. 新增 ``tools/run_mips_tests.sh``，使用当前 ``test/`` 目录中的公开样例生成汇编并运行 SPIM。

6. 新增 ``test/proj4-5-3-6-...`` 样例文件，文件名中的 ``5-3-6`` 对应 ``Project_4.pdf`` 的必做样例小节。

7. 删除与当前目录演示无关的旧批量测试、示例和报告目录，使实践四目录只保留当前演示所需材料。

4.4 主要数据结构和函数功能

4.4.1 MIPS 后端数据结构

``src/mipsgen.c`` 中使用以下结构保存 IR 和栈帧信息：

- ``Var``：表示一个变量、临时变量、数组或结构体空间，保存名称、大小、栈帧偏移和下一个变量。
- ``Instr``：表示一条 IR 文本指令，保存指令字符串和下一条指令。
- ``Func``：表示一个函数，保存函数名、IR 指令链表、变量链表、参数列表、栈帧大小和下一个函数。

后端先读取 IR 文本，将指令按函数分组，再为每个函数收集变量和参数，最后计算栈帧布局并生成汇编。

4.4.2 IR 解析和变量收集函数

- ``trim(char *text)``: 去掉一行文本首尾空白。
- ``starts_with(const char *text, const char *prefix)``: 判断指令前缀。
- ``parse_ir(FILE *input)``: 读取 IR 文件, 遇到 ``FUNCTION f:`` 时创建函数对象, 其他行加入当前函数指令链表。
- ``collect_vars(Func *func, const char *text)``: 分析一条 IR 指令中出现的变量和临时变量。
- ``collect_assignment(Func *func, char *line)``: 分析赋值指令左右两侧, 收集操作数。
- ``collect_operand(Func *func, const char *operand)``: 从 ``x``、``&x``、``*x`` 等操作数中提取变量名。
- ``add_var(Func *func, const char *name, int size)``: 向函数变量表加入变量, ``DEC`` 指令会用更大的 size 更新变量空间。
- ``add_param(Func *func, const char *name)``: 记录函数形参, 并同时为形参建立本地栈槽。
- ``layout_frame(Func *func)``: 为函数内所有变量分配 ``$fp`` 负偏移, 并计算对齐后的栈帧大小。

4.4.3 操作数装载和存储函数

- ``load_operand(Func *func, const char *operand, const char *reg)``: 把 IR 操作数装入指定寄存器。立即数 ``#n`` 用 ``li``; 普通变量用 ``lw``; 地址 ``&x`` 用 ``$fp + offset``; 间接读 ``*p`` 用两次 ``lw``。
- ``store_var(Func *func, const char *name, const char *reg)``: 把寄存器内容写回变量对应的栈

槽。

- ``branch_op(const char *relop)``: 把 IR 关系运算符转换为 MIPS 分支指令, 例如 ``==`` 到 ``beq``、``<`` 到 ``blt``。

4.4.4 指令翻译函数

- ``emit(const char *text)``: 输出一行汇编。
- ``emit_assignment()``: 翻译 ``x := y``、``x := y + z``、``x := *p``、``*p := x`` 和 ``x := CALL f`` 等赋值指令。
- ``emit_instr()``: 翻译单条 IR 指令, 分派处理 ``LABEL``、``GOTO``、``IF``、``RETURN``、``READ``、``WRITE``、``ARG`` 和赋值指令。
- ``add_pending_arg()``: 记录函数调用前遇到的 ``ARG``。
- ``emit_call_args()``: 在真正执行 ``CALL`` 前把实参压栈。
- ``clear_pending_args()``: 函数调用完成后清空暂存实参。
- ``emit_epilogue(Func *func)``: 输出函数返回序列。``main`` 通过 `syscall 10` 退出; 其他函数恢复 ``$sp``、``$ra``、``$fp`` 后 ``jr $ra``。
- ``emit_func(Func *func)``: 输出函数标签、函数序言、参数复制、函数体指令和必要的返回处理。

4.4.5 MIPS 生成总入口

``mips_generate_from_ir(const char *ir_path, const char *output_path)`` 是实践四的核心入口，执行流程如下：

1. 打开临时 IR 文件。
2. 调用 ``parse_ir()`` 将 IR 按函数组织起来。
3. 打开输出 ``.s`` 汇编文件。
4. 输出 ``.data``、换行字符串、``.globl main`` 和 ``.text``。
5. 逐个函数调用 ``emit_func()`` 输出 MIPS32 汇编。
6. 关闭输出文件并返回状态码。

4.5 目标代码生成策略

当前后端采用朴素但稳定的栈式策略：

- 每个函数建立独立栈帧。
- ``${ra}`` 和旧 ``${fp}`` 保存在当前栈帧中。
- 局部变量、临时变量、数组和结构体展开后的存储空间都放在当前函数活动记录中。
- ``${DEC x size}`` 为数组或结构体变量保留连续字节空间。
- ``${&x}`` 生成相对 ``${fp}`` 的地址。
- ``${*p}`` 和 ``${*p} := x`` 翻译为间接 ``${lw}``、``${sw}``。
- ``${ARG}`` 按 IR 顺序暂存，``${CALL}`` 前压栈，``${CALL}`` 后由调用者回收参数空间。
- 被调函数入口按照 ``${PARAM}`` 顺序从调用者参数区读取实参，并复制到自己的局部槽位。

- `READ` 使用 syscall 5 读整数。
- `WRITE` 使用 syscall 1 输出整数，并使用 syscall 4 输出换行。

这种策略不追求寄存器利用率，但结构简单，能够稳定支持递归调用和嵌套函数调用。

4.6 构建和运行

所有命令均在当前实践目录下执行：`cd submission/proj4`

检查环境：`make check-env`

构建 MIPS 生成器：`make mips`

该命令生成：`build/cmmc`

运行公开样例演示：`make mips-demo`

也可以执行：`make mips-test`

手动查看手写样例源码、生成 MIPS 汇编并运行：

```
nl -ba test/proj4-test1.cmm
build/cmmc test/proj4-test1.cmm /tmp/proj4-test1.s
sed -n '1,160p' /tmp/proj4-test1.s
spim -quiet -file /tmp/proj4-test1.s
```

4.7 结果与结论分析

执行：`make mips-demo`

当前会运行 `Project\_4.pdf` 中的公开样例：

- `test/proj4-5-3-6-exp1-1.cmm`：5.3.6 必做样例 1，输入 `7`，输出前 7 个 Fibonacci 数。
- `test/proj4-5-3-6-exp2-1.cmm`：5.3.6 必做样例 2，输入 `7`，输出 `5040`。

- ``test/proj4-test1.cmm``: 手写样例, 输出循环调用 ``square`` 后的累加结果。

演示命令会先生成 `.s`` 文件, 再打印汇编前 120 行, 最后调用 SPIM 输出程序运行结果。公开样例覆盖了循环、读写、递归函数调用和返回值; 手写样例覆盖普通函数调用、循环和累加赋值。

本次实践完成了 C++ 到 MIPS32 汇编的目标代码生成。程序能够复用前端、语义分析和 IR 生成阶段, 在正确输入上输出可被 SPIM 执行的汇编代码。至此, 编译器已经形成从源程序到目标汇编的完整主流程。

5 编译原理课程设计报告：实践 5 中间代码优化

课程设计名称：编译原理课程设计

实践内容：实践 5 中间代码优化

5.1 目的和任务

本次实践的目标是在已经生成的中间代码基础上，对 IR 进行语义保持的优化，使代码更精简，并减少后续解释执行或目标代码生成需要处理的无效指令。

主要任务如下：

1. 实现独立 IR 优化器，输入`.ir`文件，输出优化后的`.ir`文件。
2. 支持常量传播、复制传播、常量折叠、局部公共子表达式消除和无用赋值删除。
3. 对函数调用、输入输出、控制流、数组地址和间接访问采用保守处理，保证优化前后语义一致。
4. 使用实践五 PDF 的 6.3.6 必做 IR 样例和手写样例进行验收，并用 IRSim 比较优化前后的运行输出。

5.2 目录结构

实践五目录按当前提交包独立整理，主要文件如下：

```
proj5/  
  Makefile  
  note.txt  
  course/
```

```
docs/  
src/  
test/  
tools/
```

各目录职责如下：

- ``Makefile``：实践五入口，只保留优化器相关命令，包括 ``make opt``、``make opt-demo`` 和 ``make opt-test``。
- ``course/``：保存课程 PDF、IRSim 压缩包和参考优化器压缩包。
- ``docs/``：保存实践五说明文档和本报告的 Markdown、Word 版本。
- ``src/``：保存编译器源码，其中本实践新增和重点修改的是优化器相关文件。
- ``test/``：保存当前实践五验收使用的 IR 样例。
- ``tools/``：保存环境检查、IRSim 镜像构建和批量测试脚本。

当前 ``test/`` 中的样例命名对应实践五 PDF 的章节：

- ``proj5-6-3-6-exp1-1.ir``：6.3.6 必做样例 1，局部公共子表达式消除。
- ``proj5-6-3-6-exp2-1.ir``：6.3.6 必做样例 2，局部无用代码消除。
- ``proj5-6-3-6-exp3-1.ir``：6.3.6 必做样例 3，常量折叠。
- ``proj5-test1.ir``：手写样例，用于观察常量传播、常量折叠和无用赋值删除。

命名末尾的 ``1`` 表示必做内容；若后续加入 6.3.7 选做样例，可使用 ``proj5-6-3-7-expN-0.ir``。

5.3 构建与测试方法

在 ``proj5`` 目录下构建优化器：`make opt`

生成的可执行文件为: `build/cmm-opt`

优化器的单独运行格式为: `build/cmm-opt input.ir output.ir`

单独优化手写样例的完整指令为:

```
make opt
nl -ba test/proj5-test1.ir
build/cmm-opt test/proj5-test1.ir /tmp/proj5-test1.opt.ir
sed -n '1,160p' /tmp/proj5-test1.opt.ir
diff -u test/proj5-test1.ir /tmp/proj5-test1.opt.ir
```

其中第一行用于构建优化器, 第二行用于查看原始 IR, 第三行调用优化器生成优化后 IR, 第四行查看优化结果, 第五行用统一 diff 格式查看优化前后的删改内容。也可以将输入文件

替换为任意一个 ``test/`` 下的 IR 样例, 例如:

```
build/cmm-opt test/proj5-6-3-6-expl-1.ir /tmp/proj5-expl.opt.ir
sed -n '1,160p' /tmp/proj5-expl.opt.ir
diff -u test/proj5-6-3-6-expl-1.ir /tmp/proj5-expl.opt.ir
```

批量优化当前 ``test/`` 下的样例: `make opt-demo`

批量优化并用 IRSim 比较优化前后输出: `make opt-test`

如果本机还没有 IRSim 使用的 Docker 镜像, 先执行:

```
./tools/build_irsim_image.sh
```

优化后的 IR 会输出到: `build/project5-opt/`

5.4 源码文件和函数功能

5.4.1 ``src/iropt_main.c``

该文件是优化器的命令行入口。``main`` 函数检查参数数量, 要求输入文件和输出文件两个参数, 然后调用 ``ir_optimize_file(input, output)``。因此实践五可以不重新解析 C-- 源文件, 直接

以文本 IR 作为输入进行单独测试。

5.4.2 `src/iropt.h`

该头文件声明优化器对外接口：

```
int ir_optimize_file(const char *input_path, const char *output_path);
```

其他模块只需要包含该头文件即可调用优化器。

5.4.3 `src/iropt.c`

该文件是实践五的核心实现，主要功能分为以下几类：

- IR 行管理：`IrLine`、`LineVec`、`line\_push`、`read\_lines`、`write\_lines` 负责读取、保存和写回文本 IR。

- 字符串和解析辅助：
`trim`、`starts\_with`、`parse\_assign`、`parse\_binary`、`parse\_simple\_operand` 用于识别赋值、二元表达式和操作数。

- 使用和定义分析：`line\_uses\_var`、`line\_defines\_var`、`collect\_line\_use\_def` 识别每行 IR 读取和定义了哪些变量。

- 传播表维护：`Binding`、`BindVec`、`bind\_get`、`bind\_set`、`bind\_kill` 保存基本块内的常量和复制关系。

- 表达式表维护：`ExprEntry`、`ExprVec`、`expr\_find`、`expr\_add`、`expr\_kill` 保存基本块内已经计算过的二元表达式。

- 表达式规范化：`normalize\_expr` 对加法和乘法的操作数排序，使 `a + b` 与 `b + a` 能识别为同一表达式。

- 指令改写：
`substitute\_operand`、`eval\_binary`、`rewrite\_assignment`、`rewrite\_non\_assignment`、`rewrite\_pass` 实现常量传播、复制传播、常量折叠和局部公共子表达式消除。

- 局部无用赋值删除：`assignment\_has\_side\_effect`、`can\_remove\_assignment`、`dce\_pass` 删除基本块内被覆盖且未使用的赋值。

- 全局无用赋值删除：`line\_successors`、集合操作函数和 `global\_dce\_pass` 建立行级控制流后继关系，进行活跃变量分析，删除结果不再活跃且无副作用的赋值。

- 总控入口：`ir\_optimize\_file` 读取输入 IR，循环执行改写和删除过程，最后写出优化后的 IR。

5.5 修改和添加内容

相对于前一阶段仅生成 IR 的代码，本实践主要增加和修改了以下内容：

1. 新增 `src/iropt.c`，实现独立文本 IR 优化器。
2. 新增 `src/iropt.h`，提供优化器接口声明。
3. 新增 `src/iropt\_main.c`，提供 `build/cmm-opt` 命令行程序入口。
4. 修改 `Makefile`，增加 `make opt`、`make opt-demo`、`make opt-test`，并移除与当前实践无关的验收入口。

5. 新增 `test/` 下的实践五 IR 样例，按 6.3.6 必做章节编号命名，并补充手写样例。
6. 修改 `tools/run\_opt\_tests.sh`，使其只读取当前 `test/\*.ir`，生成 `build/project5-opt/\*.opt.ir`，并可用 IRSim 对比优化前后的运行输出。
7. 保留 `tools/build\_irsim\_image.sh`，用于构建 IRSim 运行所需 Docker 镜像。
8. 将报告和说明文档整理到 `docs/`，删除旧的测试目录和报告目录引用。

5.6 优化方法

5.6.1 常量传播和复制传播

优化器按基本块扫描 IR，在块内维护变量到已知值的映射。例如 `v1 := #10` 会记录 `v1` 的值为 `#10`，后续使用 `v1` 的位置可以替换为常量。对于 `v2 := v1`，如果 `v1` 已经有已知值，也会继续传播。

5.6.2 常量折叠

当二元表达式的两个操作数都是常量时，优化器直接计算结果。例如：

```
t1 := #10 + #20
```

可优化为：

```
t1 := #30
```

除法只在除数非零时折叠，避免改变运行时行为。

5.6.3 局部公共子表达式消除

同一基本块内，如果再次出现已经计算过的二元表达式，且相关变量没有被重新定义，则复用已有结果。对加法和乘法，优化器会先规范化操作数顺序，因此交换律形式也能被识别。

5.6.4 无用赋值删除

局部删除阶段会删除被后续定义覆盖、且覆盖前未被使用的赋值。全局删除阶段会进行活跃变量分析，如果一条赋值的左值在后续已经不再活跃，并且该赋值没有副作用，则删除该行。输入输出、函数调用、返回、参数传递、间接写和控制流指令都不会被删除。若变量被取地址，优化器会保守地保留相关写入，避免间接访问导致语义错误。

5.7 结果与结论

执行：`make opt-demo`

可以看到 ``test/`` 中 4 个 IR 样例都能生成优化后文件。执行：`make opt-test`

脚本会先优化当前样例，再使用 `IRSim` 分别运行原始 IR 和优化后 IR，并比较两者输出是否一致。当前结果为：

```
irsim opt tests: 4 passed, 0 failed, saved 54 IR lines
```

该流程验证了当前优化在样例范围内保持程序语义。

本次实践完成了一个保守的 IR 优化器。它能够处理常量传播、复制传播、常量折叠、局部公共子表达式消除和无用赋值删除，并通过当前实践五样例验证优化前后输出一致。后续若继续扩展 6.3.7 选做内容，可以在现有框架上加入跨基本块公共子表达式消除、常量传播、

全局无用代码删除、循环不变式外提和强度削弱。

6 结果与结论分析

本次五个编译原理子实验完成了 C--语言编译器全流程搭建：

词法分析用 Flex 精准识别各类 token，过滤无效字符，词法识别准确率达标；

语法分析依托 Bison 完成语法校验与语法树输出，错误恢复机制提升了编译器健壮性；

语义分析通过符号表与类型系统，完成变量、函数等实体的语义约束检查，提前排查逻辑错误；

中间代码生成遍历 AST 输出规范的三地址代码，为后续环节筑牢基础；

目标代码生成将中间代码转化为可在 SPIM 运行的 MIPS32 汇编指令，实现函数栈帧等核心机制。

各环节衔接顺畅，验证了编译全流程的核心逻辑，既巩固了理论知识，也完成了从源码到可执行汇编的完整编译链路落地。